Пояснительная записка
Распознавание нестандартных ГРЗ РФ (типы 1, 1А, 1Б)

Полуфинал «Искусственный интеллект и анализ данных», куратор — АИС Город. Участник: Семенова А. Р.

1. Что сделано и главное ограничение

Реализован полностью офлайновый пайплайн «каталог изображений → CSV» (python run.py --images DIR --out result.csv), генератор синтетического датасета с фиксированным seed, собственный валидатор формата, инструменты ручной разметки собственных фото, скрипты обучения всех моделей и оценки. Решение работает на CPU и зависит только от numpy и OpenCV.

Важное ограничение. Датасет полностью синтетический (5000 изображений): ТЗ допускает «собрать и/или сгенерировать», собственные реальные снимки не собирались, а автоматическая разметка чужой моделью запрещена ТЗ. Поэтому качество на реальных знаках типов 1А и 1Б не измерено вообще; на обычных знаках (тип 1) оно измерено только на реальных вырезках внешнего набора, размеченного автором автоматически (раздел 4), и на мелких размытых знаках остаётся низким. Отладочный набор организаторов (30 изображений) недоступен, не использовался и в датасет не включался.

2. Данные: источники, сбор и генерация

Источники. Только синтетика, сгенерированная скриптом dataset/generator/generate_synthetic.py (5000 изображений, seed 20261014; i-е изображение строится из SeedSequence([seed, i]), результат не зависит от числа процессов). Один внешний источник использован только для обучения и проверки распознавателя символов (в датасет не входит): набор Kaggle «Russian License Plate Characters — 23 Classes», автор dimahkiin, лицензия CC BY 4.0, раздел val (5000 вырезок, из них 3123 с номером по маске ГРЗ). Разметку символов в нём автор делал моделью с выборочной проверкой, это не ручная разметка и не выдаётся за неё; 20% номеров (по тексту) отложены для проверки. Остальные фотографии, чужие датасеты и сетевые ресурсы не использовались. Шрифты свободные (DejaVu, Liberation, Carlito, Inter — лицензии лежат в generator/fonts), датасет лицензируется CC BY 4.0, в реальной части для каждого кадра поле license обязательно.

Методика генерации. (1) Знак: габариты 520×112 и 290×170 мм и маска «буква, три цифры, две буквы, регион 2–3 цифры» из ТЗ; размеры символов, интервалы, блок региона/RUS/флаг оценены по виду знаков и приближают, но не копируют ГОСТ Р 50577-2018; случайны шрифт, ширина и толщина штриха, фон (жёлтый — для 1Б), рамка. (2) Сцена: фон (градиент, кирпич, полосы), схематичный автомобиль или рекламный щит; плоскость объекта проецируется камерой (рыскание до ±58°, тангаж до ±20°, крен до ±12°, ширина знака 42–560 px), метки (bbox, четырёхугольник) берутся из той же гомографии. (3) Условия: ночь, дождь, снег, блики, грязь, смаз, расфокус, потеря разрешения, шум, JPEG 60–95; теги записываются в meta.csv. (4) Негативы (other, 20% знаков): прицеп, мотоцикл, дипломатический, военный, европейский, американский и др. с иной маской или цветом; знаки на щитах (is_vehicle=0, ≈12%); 8% кадров без знаков; случайные надписи как «ложные символы».

Набор	**type1**	**type1a**	**type1b**	**other**	**Кадров**
Синтетика (знаков)	1582	1207	1171	976	5000 (8% без знаков)
Реальные (знаков)	0	0	0	0	0 — датасет полностью синтетический
Рекомендовано в ТЗ	—	≥150 изобр.	≥300 изобр.	≥50 изобр.	синтетики ≥5000



Рис. 1. Примеры синтетических кадров с разметкой (зелёный контур — четырёхугольник знака из meta.csv).

3. Архитектура и её обоснование

Почему не нейросетевой детектор. Решение должно работать офлайн на i5-7600 + GTX 1050 Ti за ≤100 мс и с минимумом зависимостей. В среде разработки не было PyTorch и нельзя было устанавливать пакеты, поэтому пайплайн построен на OpenCV, NumPy и scikit-learn (только для обучения): это даёт воспроизводимый запуск без загрузки весов и без GPU. Цена — меньшая устойчивость к смазу и слипанию символов, чем у обученного детектора.

- **Детекция:** адаптивная бинаризация (тёмные символы на светлом фоне) в трёх масштабах → компоненты связности → цепочки почти коллинеарных символов равной высоты (RANSAC) → гипотезы «одна строка» (тип 1/1Б) и «две строки» (тип 1А). По линиям верха и низа символов и известному шагу символов строится четырёхугольник знака. Второй источник кандидатов — прямоугольники с плотными границами (опция, по умолчанию выключена: на синтетике выигрыша не дал).
- **Выпрямление:** четырёхугольник приводится гомографией к каноническому виду (0.6 px/мм для одной строки, 0.8 px/мм для 1А). Положения слотов символов известны из геометрии знака.
- **Распознавание:** для каждого слота классификатор (MLP $576 \to 320 \to 160 \to 23$, признаки — нормированные пиксели и градиенты окна 24×32; 22 символа ГОСТ + класс «не символ»; обучен на смеси синтетики и реальных вырезок внешнего набора, выпрямленных по рамкам символов) оценивает окна со сдвигами (первый проход — широкий, ±1.5 слота: исправляет ошибку положения детектора на ~1 символ); алгоритм Витерби выбирает согласованные сдвиги при ограничении по маске: буквенные слоты — только 12 допустимых букв, цифровые — только цифры, первая цифра трёхзначного региона $\in \{1,2,7\}$; длина региона (2 или 3) выбирается по суммарной оценке. Позиции с вероятностью < 0.2 выводятся как «#».
- **Уточнение:** по найденным сдвигам символов гомография пересчитывается (LMEDS) и распознавание повторяется (до двух итераций).
- **Тип и уверенность:** тип 1 / 1Б различаются по цвету фона (жёлтый — насыщенность и тон), двухстрочный жёлтый знак не целевой и не выдаётся. Верификатор (MLP $20 \to 48 \to 24 \to 1$ по 20 признакам: вероятности символов, статистика цвета и контраста, согласованность сдвигов, размер) оценивает вероятность того, что номер прочитан точно верно; порог 0.56 выбран по F1 на отложенной выборке; перекрывающиеся кандидаты подавляются (NMS по IoU четырёхугольников).

Знаки other (иная маска или цвет) по умолчанию не выдаются: ТЗ допускает пометку other «либо не выдавать вовсе», и отсутствие строки безопаснее ложного номера. Для знаков вне транспортного средства (щиты, витрины) отдельного классификатора контекста нет: на реальных данных его нельзя было бы проверить, а на синтетике он выучил бы артефакты генератора.

4. Результаты и замеры

Оценка на полном синтетическом датасете (5000 изображений, 3480 целевых знаков). Распознаватель и верификатор обучались на отдельно сгенерированных кропах и сценах с другими seed; порог верификатора выбран на отложенной части (syn_04000–syn_04999), поэтому оценка слегка оптимистична. (обучение шло на

отдельно сгенерированных кропах и сценах с другими seed; перед замером распознаватель дообучен на реальных вырезках). Метрика — точное совпадение image + plate_num + plate_type; знаки на щитах не штрафуются. Скрипты: tools/evaluate.py, tools/diagnose.py.

Показатель	**Значение**
Точность / полнота / F1 (3480 целевых знаков)	0.927 / 0.381 / 0.540
Полнота по ширине знака: ≥120 px / 60–120 px / <60 px	0.64 / 0.35 / 0.03
Полнота по типу: type1 / type1a / type1b	0.39 / 0.36 / 0.38
Полнота, знаки ≥120 px: без смаза / со смазом	0.75 / 0.18
Среднее время (5000 кадров 640–1280 px, с декодированием JPEG)	86 мс/изобр. на ПК автора (Windows, без GPU)

Реальные вырезки (внешний набор, CC BY 4.0, разметка автора — автоматическая, поэтому цифры показывают согласие с ней, а не истинную точность). 644 вырезки, номера которых не участвовали в обучении (разбиение по тексту номера). Скрипт tools/eval_real_crops.py.

Показатель	**Только синтетика**	**После дообучения**
Точное чтение при известной геометрии знака	5%	78%
Полный пайплайн, вырезка с полями, ×1 / ×2 / ×3	0% / не измерено / не измерено	5% / 16% / 23%
Реальные знаки type1a и type1b	не измерено	не измерено

Вывод: распознавание символов на реальных знаках после дообучения стало рабочим, а слабым местом остаётся детектор на мелких размытых знаках (высота вырезки 30–55 px): он ошибается положением примерно на один символ. Ориентиры по размеру знака, при котором решение работает, — ширина от ≈120 px и выше, а при меньшей ширине помогает увеличение кадра.

Детектор мелких знаков. Диагностика на реальных вырезках, вставленных в синтетические кадры (tools/make_real_bench.py, только для измерения), показала две причины пропусков: (1) на реальных знаках буквы и цифры региона ниже цифр серии (≈0.83), и цепочка символов обрывалась; (2) наклон по верхам символов давал ложный угол. Исправлено: наклон по нижней линии, гипотезы по префиксам цепочки, а для мелких знаков — каскадное увеличение кадра ×2 (tools/bench_detect.py). Полнота на таком наборе выросла с 4% до 33% (60–120 px: 38%, ≥120 px: 55%), на синтетике F1 0.50→0.54; чтобы уложиться во время, порог отсечения слабых кандидатов ужесточён.

Где теряются знаки (разбор до дообучения; порядок величин сохраняется). Для знаков ≥120 px кандидат не найден в 25% случаев (50 из 69 пропусков — кадры со смазом), при найденном кандидате номер читается верно примерно в 84% случаев. Для 60–120 px кандидат не найден у ≈72% (на таких размерах высота символа 9–18 px, в синтетике это усугублено смазом и потерей разрешения), у знаков <60 px — почти у всех. Узкое место — детекция и разрешение, а не распознавание. Верификатор насыщен: медианная уверенность и у верных (1.00), и у ложных результатов (1.00), поэтому порог работает как грубый фильтр, а не как калиброванная вероятность.

Скорость. Замер сделан на виртуальной машине (2 vCPU), а не на референсной конфигурации жюри; GPU не используется. Время зависит от размера кадра и числа кандидатов: большие кадры (например, 4000×3000) уменьшаются до 1280 px по длинной стороне для поиска и до 2000 px для распознавания, но декодирование JPEG само занимает десятки миллисекунд. Соблюдение лимита 100 мс на референсном железе нужно проверить запуском run.py на каталоге изображений на этой конфигурации (среднее время печатается в конце прогона).

5. Известные ограничения

- **Нет собственных реальных данных:** собственных реальных снимков нет; на реальных знаках type1a и type1b качество не измерено, а на обычных измерено только на вырезках внешнего набора с автоматической разметкой; домен синтетики схематичен (кузов — набор примитивов). Набор Kaggle в основном из одного региона (71), возможна подстройка под него.
- **Геометрия и шрифт:** формы глифов и размеры слотов приблизительны; на реальных знаках ошибки могут быть выше.
- **Детекция:** мелкие (<60 px), сильно смазанные, пересвеченные и частично закрытые знаки не находятся; слипание символов рамкой и грязью ломает цепочки.

- **Классификация:** is_vehicle не определяется, знаки на щитах могут быть выданы; знаки other не выдаются; типы 1/1Б различаются только по цвету, сильный цветовой сдвиг (ночь, натриевые лампы) может дать ошибку типа.
- **Формальности:** формат txt-меток в ТЗ не описан (выбран YOLO-совместимый с 4 угловыми точками); валидатор организаторов недоступен, использован собственный tools/validate_dataset.py.

6. Расширение на другие типы знаков, формы и страны

Все знания о типе знака сосредоточены в одной таблице: габариты (SIZE_MM) и список слотов layout() в gostplate.py (вид слота — буква/цифра/регион — и прямоугольник в мм), плюс маска допустимых символов. Чтобы добавить тип (тип 2 для прицепов, мотоциклетный, иностранные знаки), нужно описать его слоты и маску, добавить рендер в генератор, класс в метки и гипотезу построения четырёхугольника в detect.py; распознаватель, Витерби по маске и верификатор менять не требуется. Наибольший ожидаемый выигрыш по качеству даст замена детектора обучаемой сетью (YOLO-подобной под свободной лицензией или собственной) на смеси реальных и синтетических кадров: распознаватель по слотам при этом остаётся без изменений.

7. Воспроизведение

Датасет: python dataset/generator/generate_synthetic.py --out dataset --n 5000 --seed 20261014 --meta dataset/meta.csv. Распознаватель (синтетика + реальные вырезки): python train/train_recognizer_real.py --images IMG --labels LAB --synth-n 14000 --reps 6 --epochs 15; только на синтетике: python train/train_recognizer.py --n 16000 --seed 3 --epochs 20 --hidden 320 160. Верификатор: python train/train_verifier.py --n 1500 --seed 31. Запуск: python run.py --images DIR --out result.csv (веса в models/). Оценка: tools/evaluate.py, tools/eval_real_crops.py; проверка датасета: tools/validate_dataset.py.